# Selecting Reliable LLM Reasoning: group-centred training and answer consensus

DASE7506 · Advanced Machine Learning Methods · Fall 2026 · MP2 Student ID: **3036837382** · Protocol `dase7506-mp2-open-hard-verifier-v5` Chosen question: **"Should a solution be judged on its own?"** Do group-centred training and same-question answer consensus improve candidate selection?

---

## 1. Problem, metric and hypotheses

A language model produced four candidate solutions to each mathematics question. Exactly one integer index (0-3) must be returned per question, in bank order. The official scorer parses the selected candidate's **final numeric answer** with `common.parse_answer` (the `####` tail, else the last `\boxed{...}`) and compares it with the parsed reference; equivalent fractions and decimals count as correct, and **any** correct candidate earns the point. Reasoning steps are never graded. Success is therefore *selection accuracy in percent* on the complete test bank.

Because grading only inspects the final answer, the whole task reduces to one question: **given four candidates, which one's final answer equals the reference?** The experiments address three hypotheses:

- **H1** - The frozen whole-trace representations carry useful correctness signal, so a light supervised model beats popularity-style baselines.
- **H2** - Group-centred representations and answer-consensus features improve over the tested absolute-vector training scheme. The role of centring in linear ranking requires separate interpretation.
- **H3** - Answer-consensus (majority voting) helps, but fails systematically when the majority is wrong or when several answers tie.

A negative comparison can still be informative. The saved configurations define the rules used below, but the available Git history does not independently establish when all rules were fixed or when the test bank was first inspected. The current audit found no reference-answer input in the prediction path. These are evidence limits, not proof of the full historical sequence. The final model and its reported test result remain frozen during the corrections and validation-only follow-ups.

## 2. Data and protocol

Official supply: 4,755 / 798 / 467 train/validation/test questions, four candidate traces each, one `float32 [N,4,1024]` whole-trace embedding per candidate (written on disk as `[4N,1024]` and reshaped by `common.load_inputs`). `python check_data.py` passes: hashes, split disjointness, feature/reference alignment, embedding shape `[4755|798|467, 4, 1024]`. The upstream commit is `bb801f4f2e7629d270df765ab12c333b2dc616bb` (2026-10-12) and all 20 files of `PACKAGE_MANIFEST.json` still hash to their listed digests - the official data, parser and scorer were not modified.

**Labels.** For every training question *q* and candidate *j*, `y[q,j] = 1` iff `parse_answer(candidate)` is not `None` **and** equals `parse_answer(reference)`. A question may have several correct candidates; the current implementation keeps every positive. 4,755 questions give 19,020 candidate records, but the experimental unit stays the question. A parse failure is not an answer and is never pooled with another failure - multiple `None`s are not treated as agreement.

**What the data looks like** (Phase-2 profile, `runs/phase2_stats.md`):

| statistic | train | validation |
|---|---|---|
| mean correct candidates / question | 3.0185 | 3.0276 |
| questions with all four correct | 2,242 (47.1 %) | 379 (47.5 %) |
| questions with ≥2 distinct valid answers | 2,114 (44.5 %) | 355 (44.5 %) |
| parseable candidates / question (1/2/3/4) | 27 / 81 / 628 / 4019 | 3 / 25 / 98 / 672 |
| positive rate over candidate records | 0.7546 | 0.7571 |

Two consequences drive the design. (i) Nearly half of all questions have four correct candidates, so any index scores on that subset. Uniform random therefore has an expected validation accuracy of 75.69 %; the saved seed-42 draw achieves 76.82 %, rather than the 25 % appropriate to one-positive four-way classification. (ii) The main

discrimination problem lies in the ~45 % of questions where parsed answers disagree, and a candidate that ends without a readable `####` or `\boxed{...}` tail - or whose box contains a nested brace such as `\boxed{\frac{1}{2}}` - is unparseable and can never earn the point even when its number is right (4.6 % of train records, 4.9 % of validation records).

## 3. Method

### 3.1 Fixed baselines

| id | rule |
|---|---|
| B0 | uniform random over 0-3, seed 42 (reported together with the mean correct-candidate ratio as the expected random accuracy) |
| B1 | always candidate 0 (does not search for the best position) |
| B2 | majority vote over valid parsed answers only; tie → smallest original index; all failures → 0 |
| B3 | first candidate whose answer parses; all failures → 0 |

### 3.2 Features (input-only)

Nothing in the feature path reads a reference answer or a question id; each feature is computed from `question`, the four candidate texts and the frozen embeddings. Six groups:

- **l2** - candidate embedding, L2-normalised (raw row norms are ≈57, so normalisation matters).
- **raw** - unnormalised embedding, as a control.
- **rel** - *group-centred* vector: the candidate's normalised embedding minus the mean of its four same-question candidates. This removes the shared mean before training. For the final linear scorer the shared mean cancels in within-question ranking; it can still change the fitted model.
- **cos** - mean and maximum cosine similarity of the candidate to the other three.
- **vote** - five consensus features from the parsed answers of the same question: this answer's vote count, number of distinct valid answers, whether the candidate is in a top-vote group, whether the answer parses at all, and how many candidates parse.
- **surf** - ten surface features (characters, lines, digits, `\boxed`, `####`, `=`, LaTeX markers, number tokens) used as an explicit "does formatting already explain the score?" control.

All active feature blocks are standardised with means and standard deviations fitted on the training part only and stored in the inference asset; a near-zero standard deviation is replaced by 1.

### 3.3 Models

- **M1/M2/M3 linear** - one logistic-regression score per candidate, binary cross-entropy, all positives kept; the four candidates share one parameter vector, so the model never treats the candidate position as a feature. Prediction is the across-candidate `argmax`.
- **Small MLP** - one hidden layer, widths 64 and 128, trained from scratch, no external weights.
- **Group multi-positive loss (optional, §4.4)** - `L(q) = −log Σ_{j∈P(q)} softmax(s_q)[j]`, implemented in NumPy with Adam. Questions whose candidates are all positive have zero data gradient.

Selection used question-level accuracy on validation, rather than per-candidate classification accuracy. The saved search includes more fits than the initial approximate 24-fit planning budget. Its actual scope and costs are reported in §6; the repeated use of validation is a limitation of model selection.

## 4. Results

### 4.1 Baselines vs the frozen model (official scorer)

| method | validation acc % | test acc % |
|---|---|---|
| B0 uniform random (seed 42) | 76.817 | 49.679 |
| B1 always candidate 0 | 78.195 | 49.679 |
| B2 majority vote | 86.216 | 69.165 |
| B3 first parseable candidate | 81.704 | 52.677 |
| **Frozen model (rel + vote, C = 0.002)** | **91.604** | **79.657** |

The frozen model is 731/798 on validation and 372/467 on test (student ID 3036837382 submits **79.6574**; `release/test-score.json`). The test distribution is much harder - majority voting drops from 86.2 % to 69.2 %, consistently with the guide's "≥2 distinct valid answers" guarantee - but the model's margin over majority voting is *larger* there (+10.5 points) than on validation (+5.4). H1 holds: the frozen representations alone take a plain logistic regression well past the strongest fixed rule.

### 4.2 Feature-combination comparisons (validation, best `C` per row)

| features | dim | val acc % | Δ vs full |
|---|---|---|---|
| surface only | 10 | 81.203 | −10.401 |
| vote only | 5 | 86.216 | −5.388 |
| absolute vector only (l2) | 1024 | 89.724 | −1.880 |
| l2 + cos | 1026 | 89.975 | −1.629 |
| abs + rel (no vote) | 2048 | 90.727 | −0.877 |
| l2 + vote | 1029 | 90.476 | −1.128 |
| l2 + rel + cos + vote | 2055 | 91.479 | −0.125 |
| **rel + vote (frozen)** | **1029** | **91.604** | **0.000** |
| rel + cos + vote | 1031 | 91.604 | 0.000 |
| rel + vote + surface | 1039 | 91.604 | 0.000 |

The tested surface features alone reach 81.2 %, and appending them to the final features does not increase validation accuracy (91.604 → 91.604). This establishes no extra benefit from these explicit features; it **does not exclude formatting information already encoded in the frozen vectors**. The group-centred training scheme plus consensus reaches 91.604 %, versus 89.724 % for the tested absolute-vector scheme and 90.476 % for absolute vectors plus consensus. These are feature-combination comparisons, with `C` selected per row. Switching from `rel+vote` to `l2` also changes the representation, so it is not a direct "remove vote" ablation.

For a linear score, let `u_j` be the L2-normalised candidate vector, `m_q` the same-question mean, and `D` and `μ` the training-fitted scale and mean. Then $s_j = w^\top D^{-1}(u_j - m_q - \mu) + v^\top vote_j + b$. The terms involving `m_q`, `μ` and `b` are shared by all four candidates and cancel in their score differences. Centring therefore **does not expand the linear ranking function family** or establish that the final vector score dynamically compares rival traces at prediction time. Its observed benefit may arise from the candidate-level training objective, fitted scales and effective regularisation. Holding the fitted weights and vote features fixed, replacing centred vectors by absolute vectors preserves all 798 validation choices in the audit (maximum pairwise score-difference error below `9e-15`). Vote features do explicitly use the other candidates' answers. H2 is supported as an empirical training-scheme comparison, with this restricted mechanism interpretation.

**Direct removals at the frozen settings.** To separate a true ablation from the best-`C` comparisons above, each remaining feature group was retrained at `C=0.002`, seed 42, with its own training-only standardisation. These

controls were run **after the test results were already known**, using only train/validation inputs and references. They explain the existing method and were not used to reselect or replace the final model.

| features, fixed C=0.002 | dim | train selection % | validation correct / 798 | val acc % |
|---|---|---|---|---|
| rel + vote (frozen reference) | 1029 | 92.2397 | 731 | 91.6040 |
| rel only (remove vote) | 1024 | 92.8917 | 725 | 90.8521 |
| vote only (remove rel) | 5 | 85.2576 | 688 | 86.2155 |

Adding vote to the fixed-setting relative-vector fit yields a net six more correct questions (+0.7519 points); adding the vector features to vote-only yields 43 (+5.3885 points). This is specific to these settings and validation questions, without an independent post-selection guarantee. Logs: `runs/corrections/exp_rel_only_C0.002_s42.json` and `exp_vote_only_C0.002_s42.json`.

### 4.3 Regularisation and deterministic repeatability

The fit is deterministic: `lbfgs` on a fixed, convex objective with fixed features converges to the same solution for every seed, so all three seeds reproduce identically. Accuracy as a function of `C` for the frozen feature set:

| C | val acc % (seeds 17 / 42 / 2026) |
|---|---|
| 0.001 | 91.353 / 91.353 / 91.353 |
| 0.0015 | 91.604 / 91.604 / 91.604 |
| **0.002** | **91.604 / 91.604 / 91.604** |
| 0.003 | 91.479 / 91.479 / 91.479 |
| 0.005 | 91.353 / 91.353 / 91.353 |

Strong regularisation is useful in this search: training selection accuracy falls from 92.2 % (`C=0.002`) to 91.2 % at `C=0.0003` and rises to 93.4 % at `C=0.01` while validation peaks near `C=0.002`. The plateau (`C` 0.0015-0.002) shows a local region with equal validation accuracy. Changing `random_state` does not change this deterministic `lbfgs` optimisation path: the zero standard deviation establishes repeatability, **not independent evidence of generalisation stability**, and does not remove validation selection bias.

### 4.4 Nonlinear and group-loss alternatives (validation)

| method | val acc % | note |
|---|---|---|
| MLP hidden 64 | 91.604 | adam, 400 iters; training selection accuracy 100 % |
| MLP hidden 128 | 91.228 | training selection accuracy 100 % |
| original group-loss run | 90.852 | loss/gradient implementation discrepancy; not a valid test of the stated objective |
| corrected group-loss control | 90.852 | 725/798; best checked iteration 50, training selection 96.320 % |
| **linear logreg (frozen)** | **91.604** | reference |

The MLP reaches 100 % **training question-selection accuracy**: it selects a positive for every training question. This is not candidate-level classification accuracy and does not show that all 19,020 candidate labels were memorised. Neither tested width exceeds the linear model on validation, so the submitted model stays linear.

The audit found a discrepancy in the original group-loss implementation: the loss was averaged over `N` questions but its data gradient divided by `4N` candidate rows, while the L2 gradient was unchanged. The saved 90.852 % result is retained as an audit record, **not as evidence against the stated loss**. The corrected gradient passes seven checks including finite differences (maximum absolute error below `6e−11`). The follow-up holds `λ=0.001`, learning rate 0.05, 1,500 iterations and seed 42; it reaches 725/798 at checked iteration 50, versus iteration 950 in the original run. The identical headline accuracy is coincidental and does not validate the old implementation. This corrected configuration still falls six questions below the frozen binary-loss model, but one configuration does not establish that the group objective is intrinsically inferior. All-positive groups do have zero data gradient (2,242 training questions); this alone does not explain performance. Like the direct ablations, this is a post-test-result, validation-only diagnostic and leaves the submitted model unchanged. Full trajectory:
`runs/corrections/exp_group_relvote_corrected.json`.

## 5. Grouped evaluation and failure analysis

### 5.1 Subsets (validation)

| subset | n | model % | majority % |
|---|---|---|---|
| only one distinct valid answer, possibly with parse failures | 443 | 100.0 | 100.0 |
| ≥2 distinct valid answers ("hard") | 355 | **81.13** | **69.01** |

All observed gains over majority voting occur inside the hard subset. On the other 443 questions both methods achieve 100 %, but this subset is not safe for any choice: it can contain unparseable candidates, which still score zero. The truly all-four-correct subset contains 379 questions. Against majority voting the paired, question-level bootstrap (5,000 resamples, seed 17) gives **Δ = +5.39 points, 95 % CI [3.51, 7.39], P(better) = 1.00**; against "always candidate 0" it is +13.41 [10.65, 16.29]. Bootstrap intervals describe this validation set only and do not erase the selection effect of repeated comparisons. The local `C` plateau in §4.3 is descriptive and provides no correction for that selection effect.

### 5.2 Vote-pattern breakdown (validation)

| vote pattern | n | model % | majority % |
|---|---|---|---|
| 4 identical | 379 | 100.00 | 100.00 |
| 3+1 | 151 | 87.42 | 85.43 |
| 3 + 1 fail | 50 | 100.00 | 100.00 |
| 2+2 | 34 | **85.29** | **41.18** |
| 2+1+1 | 88 | 75.00 | 61.36 |
| 2+1 + 1 fail | 28 | 85.71 | 85.71 |
| 1+1+1+1 | 20 | 65.00 | 25.00 |
| 1+1+1 + 1 fail | 20 | 65.00 | 55.00 |
| others (≥2 fails) | 28 | - | - |

The pattern is exactly what H3 predicts. Where one answer dominates (3+1, 4) majority voting is already good and the model adds little. Where the vote is tied, majority voting is forced onto the smallest index and performs poorly: **2+2 reaches 41.2 %** while the model reaches 85.3 %, and four-valid-answer groups reach 25 % (chance among four) while the model reaches 65 %. An important part of the advantage is tie resolution, although improvements also occur outside tied groups.

### 5.3 Error cases (fixed sampling rule, seed 42)

| category | n |
|---|---|
| majority wrong, model **right** ("fixed") | 54 |
| majority right, model **wrong** ("broken") | 11 |
| both wrong | 56 |

Cases are grouped by the two methods' saved outcomes, with seed-42 sampling represented in the analysis artefacts. These examples illustrate successes and failures; they do not identify a causal mechanism:

- **Fixed - tie broken by sign (validation q419).** Parsed answers `−25, −25, 25, 25`; the reference is `25`. Majority voting must fall back to the smallest index and returns `−25`; the model selects a `25`. This shows a successful choice in a sign-related tie, without proving that the model checks signs.
- **Fixed - 2+2 tie (q539).** Answers `20, 20, 100, 100`, reference `100`. Majority picks `20` (smallest index among two-vote winners); the model picks `100`.
- **Fixed - all-distinct (q185).** `1, 4, 3333, 22500`, reference `4`. Majority picks the first candidate; the model picks the correct one.
- **Broken - plausible wrong majority (q275).** `400, 700, 700, 400`, reference `400`. Majority breaks the 2+2 tie toward index 0 and happens to be right; the model prefers the two `700` candidates and picks a wrong one. The

scores favour a wrong answer in this tied group; the observation does not isolate which semantic or formatting cue caused that preference.

- **Broken - arithmetically wrong singleton (q29).** `9, 18, 6, unparseable`, reference `9`. The model picks `6`.
- **Both wrong (56).** In every one of the 798 validation questions at least one candidate parses

to the reference, so accuracy has a ceiling of 100 % and no error here is an impossible item - these are selection misses despite a positive being available. On 23 of the 56 the model follows majority voting to the *same* wrong index. Illustrative cases have a correct **minority** answer while both methods choose a more popular wrong answer (q: gold `19`, parsed votes `19 / 10 / 5 / 10`, both methods pick `10`; gold `40`, votes `100 / 80 / 40 / 100`, both pick `100`). This is consistent with a remaining plurality bias, although these cases do not establish its cause. The model does not simply copy majority voting: the two disagree on 2 out of 3 questions (agreement 33.3 %), and of the 110 questions where majority voting fails the model repairs 54 (49 %).

## 6. Resources, reproducibility and limits

| item | measured | limit |
|---|---|---|
| inference asset, uncompressed | **26,244 B = 0.025 MiB** (`config.json` + `linear.npz`) | 16 MiB |
| fresh prediction + independent scoring, complete test bank | **min 0.194 / mean 0.199 / max 0.209 s** | 30 s |
| independent audit rerun, prediction + scoring | **0.201 + 0.074 = 0.275 s** | 30 s |
| threads | 4 total (`OMP` / `OPENBLAS` / `MKL` / `VECLIB` / `NUMEXPR` = 4) | 4 |
| final model build | 2.430 s (0.898 s features + 1.532 s fit), CPU only | - |

Hardware: Apple M2 (8 cores), macOS 15.6, Python 3.12.15, NumPy 2.5.3, SciPy 1.18.1, scikit-learn 1.9.1. The 30-second budget is decided on the course Xeon Platinum 8457C, which I cannot measure. The local measurements are well below the limit, but do not establish eligibility on the course reference machine.

**Search cost and records.** `experiments.csv` contains 125 rows and 120 distinct run IDs, including reruns, totalling **202.061 s** of recorded fitting. The older unstandardised search is separately retained as 12 rows in `experiments_v1_unstandardized.csv`, totalling **48.337 s** of fitting. The original, discrepant group-loss run records **18.619 s**, and the final bundle fit records **1.532 s**. These four separate records total **270.549 s of recorded fitting**. Correction follow-ups add **22.428 s** for group loss, **1.676 s** for rel-only and **0.644 s** for vote-only (24.748 s total, one CPU thread per process), bringing the recorded fitting sum to **295.297 s**. These sums are not an end-to-end development wall-clock measurement. A rerun appends a CSV row but can overwrite its run-ID JSON and bundle, so a distinct run ID does not enumerate every execution. Feature extraction was reused within sweeps: adding each row's repeated `feature_seconds` would overcount its actual cost. No complete development wall-clock total is available. The correction commands, timings and unchanged final-asset hashes are in `runs/corrections/repair_controls_manifest.json`.

Contract checks (`tools/test_contract.py`, all pass): arbitrary question counts (1/5/7/100/798), Python `int` outputs in 0-3, choices in bank order, bit-identical repeats, a clear `FileNotFoundError` for a missing checkpoint, `ValueError` for `checkpoint=None` and for a shape mismatch. `predict.py` never opens references; the method never branches on IDs. A contract check exposed a reproducibility hazard that was corrected: six validation questions contain **duplicated candidate texts**, so the duplicated pair gets *exactly* equal scores (on four questions that pair sits at the top), and BLAS summation differences between differently-sized matmuls flipped the `argmax` by ~1e-15. The frozen method applies the documented tie rule - smallest original index among candidates within `1e−9` of the maximum - which is stable under changing batch size; it changes no reported accuracy (91.604 / 79.657).

**Limitations.** (i) The frozen embeddings are the only semantic signal; the model cannot verify a single arithmetic step, and the guide is explicit that the score only reflects the final answer. (ii) 47 % of train/validation questions are all-correct; separately reporting the 44.5 % validation hard subset (81.1 % vs 69.0 %) makes the remaining discrimination task visible. (iii) Repeated selection used this validation set; neither the bootstrap interval nor deterministic seed repeats corrects for it. (iv) Existing history does not independently establish the original test-access and freeze sequence. This revision preserves the submitted model, asset hashes and test score; new correction experiments are confined to training/validation analysis. (v) Frozen vectors may encode formatting, and the available controls cannot exclude reliance on those cues.

**Conclusion.** Group-centred training plus answer-consensus features, fitted with a single strongly-regularised logistic regression and no nonlinearity, raise selection accuracy from 86.2 % (majority vote) to 91.6 % on validation and from 69.2 % to 79.7 % on the harder test bank, for 26 KB of assets and approximately 0.2-0.3 s locally for fresh prediction and independent scoring. The method improves many tied-answer choices. Centring benefits the tested training scheme but cancels algebraically in the final linear ranking; the experiments do not prove semantic reasoning verification or rule out formatting signals in the vectors.

## References and disclosure

- Course supply: `guide/GUIDE.md`, `code/README.md`, protocol `dase7506-mp2-open-hard-verifier-v5`; upstream commit `bb801f4f…`. Data derived from GSM8K (MIT); candidates and representations frozen from Qwen3-0.6B. `data/LICENSE.txt`, `data/NOTICE.txt` retained; neither external data nor extra pretrained weights were used.
- **AI disclosure.** Substantive AI assistance (an LLM coding agent) was used to: implement the feature extractor, training/analysis scripts and the NumPy inference path; run the experiment sweeps; draft this report; and audit and correct the analysis and a comparison implementation. The numerical results come from saved execution logs and official scorer outputs, not invented estimates. The student is responsible for checking the submitted code, evidence and claims and confirming their understanding before submission. The frozen configuration, assets and seeds are listed in `release/resources.json`; asset hashes in `release/asset-hashes.txt`.